

CIPHERMAIL: SECURE EMAIL SYSTEM WITH AUTONOMOUS SECOPS

PHASE I REPORT

Submitted by

HEMNATH V – 230701115

HARIPRASATH R S – 230701103

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600 025

September 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled “**CIPHERMAIL: SECURE EMAIL SYSTEM WITH AUTONOMOUS SECOPS**” is the bonafide work of **HEMNATH V (230701115), HARIPRASATH R S (230701103)**, who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

SIGNATURE

Dr. Manoranjini M.E, Ph.D,
Professor and Head,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

SIGNATURE

Mrs. M. Priyachitra (SS),
Assistant Professor,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner

External Examiner

ABSTRACT

communication is widely used for exchanging personal, academic, business, and confidential information. However, conventional email systems primarily provide encryption during transmission and storage, while the email provider may still technically access message content. Users also face security threats such as phishing links, sender spoofing, and dangerous email attachments.

This project proposes **CipherMail**, a Chrome browser extension designed to provide an additional privacy and security layer for Gmail. The system combines **end-to-end PGP email encryption** with an integrated **email threat-detection mechanism**. The encryption component uses OpenPGP.js to generate cryptographic keys and perform encryption, decryption, signing, and verification directly within the user's browser. A hybrid encryption approach uses a randomly generated AES session key for message encryption and the recipient's public key to protect the session key.

The threat-detection component analyzes email links, sender information, and attachment filenames using local heuristic rules and the VirusTotal API. The results are combined into Safe, Suspicious, or Dangerous classifications. CipherMail follows a no-backend architecture in which cryptographic operations are performed inside the browser and user data is stored locally using the Chrome storage API.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavour to put forth this report. Our sincere thanks to our Chairman Mr. S. MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and sincere endeavouring in educating us in their premier institution.

Our sincere thanks to Dr. S.N.S MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to Dr. Manoranjini, Ph.D., Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, Dr. P. Muneeshwari, Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project. We thank our Project Coordinator, Mr. Karthik, B.E., M.E., Assistant Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

HEMNATH V

230701115

HARIPRASATH R S

230701103

TABLE OF CONTENT

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	ix
1.	INTRODUCTION	1
	1.1 GENERAL	2
	1.2 OBJECTIVE	2
	1.3 EXISTING SYSTEM	3
	1.4 PROPOSED SYSTEM	4
2.	LITERATURE SURVEY	6
	2.1 LITERATURE REVIEW	6
	2.2 COMPARISON AND DISCUSSION	8
	2.3 RESEARCH GAP AND CONCLUSION	9
3.	SYSTEM DESIGN	10
	3.1 SYSTEM ARCHITECTURE	10
	3.2 SYSTEM REQUIREMENTS	12
	3.3 DATA FLOW AND ACTIVITY DESIGN	15
	3.4 DEPLOYMENT AND SCALING	17
4.	PROJECT DESCRIPTION	18
	4.1 METHODOLOGIES	18

	4.2 MODULES	19
	4.3 MODULE INTERACTION	20
5.	IMPLEMENTATION, RESULTS AND DISCUSSION	21
	5.1 IMPLEMENTATION RESULTS	21
	5.1.1 PGP KEY MANAGEMENT AND ENCRYPTION	22
	5.1.2 GMAIL COMPOSE ENCRYPTION	23
	5.1.3 MESSAGE DECRYPTION AND SIGNATURE . VERIFICATION	23
	5.1.4 THREAT DETECTION AND RISK SCORING	24
	5.2 DISCUSSION	25
	5.3 TEST SCENARIOS AND PROTOTYPE STATUS	28
6.	CONCLUSION AND FUTURE WORK	32
	6.1 CONCLUSION	32
	6.2 FUTURE WORK	32
	REFERENCES	34
	APPENDICES	36

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.1	Comparison of Reviewed Literature	6
2.2	Comparison of Existing Approaches and Proposed System	8
3.1	Software Requirements	13
3.2	Hardware Requirements	14
3.3	Stored Data and Privacy Scope	14
5.1	Prototype Implementation Status	29
5.2	Test Scenarios and Expected Behaviour	30

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
1.1	Overall proposed project architecture	5
3.1	System Architecture	11
3.2	Level-1 Data Flow Diagram	15
3.3	Activity Diagram	16
5.1	CipherMail Setup	25
5.2	CipherMail Interface	25
5.3	Encrypted Email Composition Interface	26
5.4	PGP Key Unlock Interface	26
5.5	PGP Message Decryption Interface	27
5.6	Email Threat Analysis Result	27
5.7	CipherMail Chrome Extension Configuration	28

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
PGP	Pretty Good Privacy
OpenPGP	Open Pretty Good Privacy
API	Application Programming Interface
Gmail	Google Mail
URL	Uniform Resource Locator
VT	VirusTotal
AES	Advanced Encryption Standard
ECC	Elliptic Curve Cryptography
DOM	Document Object Model
MV3	Manifest V3
CSP	Content Security Policy
HTTPS	Hypertext Transfer Protocol Secure
UI	User Interface
API key	Application Programming Interface authentication key

CHAPTER 1

INTRODUCTION

Email systems have become a central part of personal, academic and organizational communication. Modern webmail services protect network traffic through HTTPS and maintain security controls around stored mail, but users may still require an additional layer of end-to-end protection when the message itself should remain readable only by the communicating parties. Public-key cryptography provides a way to achieve this by encrypting a message using the recipient's public key and decrypting it with the corresponding private key.

Email users also face a different class of risk: a message can be delivered successfully and still contain a deceptive URL, a spoofed sender identity or a dangerous attachment. Phishing messages are often designed to appear trustworthy and can exploit user attention at the moment of clicking a link or opening a file. A security mechanism placed directly in the inbox can provide additional information before that action.

CipherMail combines these two requirements in one Chrome extension for Gmail. The privacy layer adds PGP encryption, signing and decryption capabilities directly to Gmail. The threat- detection layer adds a shield action to the existing email hover toolbar and performs local heuristic analysis plus VirusTotal URL reputation checks. The complete system is designed without a dedicated CipherMail backend.

1.1 GENERAL

The project is designed around two complementary security needs. First, a user may want message content to remain confidential even from the email provider. Second, the user may want to inspect a suspicious message before interacting with its links or attachments.

The encryption layer uses OpenPGP.js 6.3.1 as the cryptographic library. The extension generates PGP identities, stores protected private-key material locally, manages contacts and inserts encryption controls into Gmail's compose interface. The cryptographic implementation is deliberately kept inside a dedicated wrapper rather than implementing cryptographic mathematics manually.

The threat-detection layer operates on the email currently being inspected. It extracts links, sender display information and attachment names. Local checks identify common warning signs. For links, the extension can query VirusTotal using the user's own API key. The resulting information is combined into a risk classification.

The project uses Chrome Manifest V3, content scripts, a background service worker, chrome.storage.local and standard HTML/CSS interfaces. This architecture separates Gmail DOM interaction from sensitive cryptographic operations and avoids the need for a custom application server.

1.2 OBJECTIVE

- Provide end-to-end PGP encryption and signing capabilities directly within Gmail.
- Generate and manage PGP identities and public-key contacts.
- Allow users to import and export keys for backup and contact exchange.

- Automatically detect PGP blocks in opened messages and provide decryption and signature status.
- Add a threat-scan shield to Gmail's email hover toolbar.
- Identify suspicious URLs using local heuristic rules and VirusTotal reputation information.
- Identify sender display-name/domain inconsistencies and risky attachment extensions.
- Present Safe, Suspicious or Dangerous results before the user takes a risky action.
- Keep CipherMail data and cryptographic operations local wherever possible.
- Operate without a CipherMail backend server or project-specific database.

1.3 EXISTING SYSTEM

Conventional Gmail provides encrypted transport and security features, but the project identifies a separate requirement for user-controlled message encryption in which the message body is protected using the recipient's public key before it is handled by the mail provider.

Conventional email workflows also depend heavily on the user's ability to recognize suspicious messages. Gmail may provide its own security warnings, but a project-specific threat-analysis action can expose the extracted URL, sender and attachment indicators in a single results panel without requiring the user to leave the inbox.

Limited awareness of the state of other buses.

- Message content is normally handled by the email provider as part of the webmail service.
- Public-key encryption is not automatically applied to ordinary Gmail messages by the user's default compose workflow.
- Users may need separate tools for PGP key management and encryption.

- Suspicious links may require separate checking workflows.
- Sender spoofing and dangerous attachment indicators are not necessarily summarized using one project-specific risk score.
- Security decisions can be difficult when threat information is separated across different tools.

1.4 PROPOSED SYSTEM

The proposed system is CipherMail, a Chrome extension that adds both privacy and threat-detection capabilities to Gmail. The privacy component provides PGP key management, encryption, signing, decryption and verification. The threat-detection component adds a shield action to each email's hover toolbar.

When a user composes an encrypted email, the extension reads the message body, obtains the recipient's public key and uses OpenPGP.js to perform hybrid encryption. The resulting PGP block replaces the plaintext message content before Gmail sends it. On the receiving side, the extension detects the PGP block and provides a decrypt action that uses the recipient's private key and passphrase.

When a user scans an email for threats, the extension extracts the message's links, sender information and attachment names. Local rules run first. Each link can then be checked against VirusTotal. The local and reputation results are combined into a Safe, Suspicious or Dangerous verdict. Warnings are shown before flagged links open or flagged attachments are downloaded.

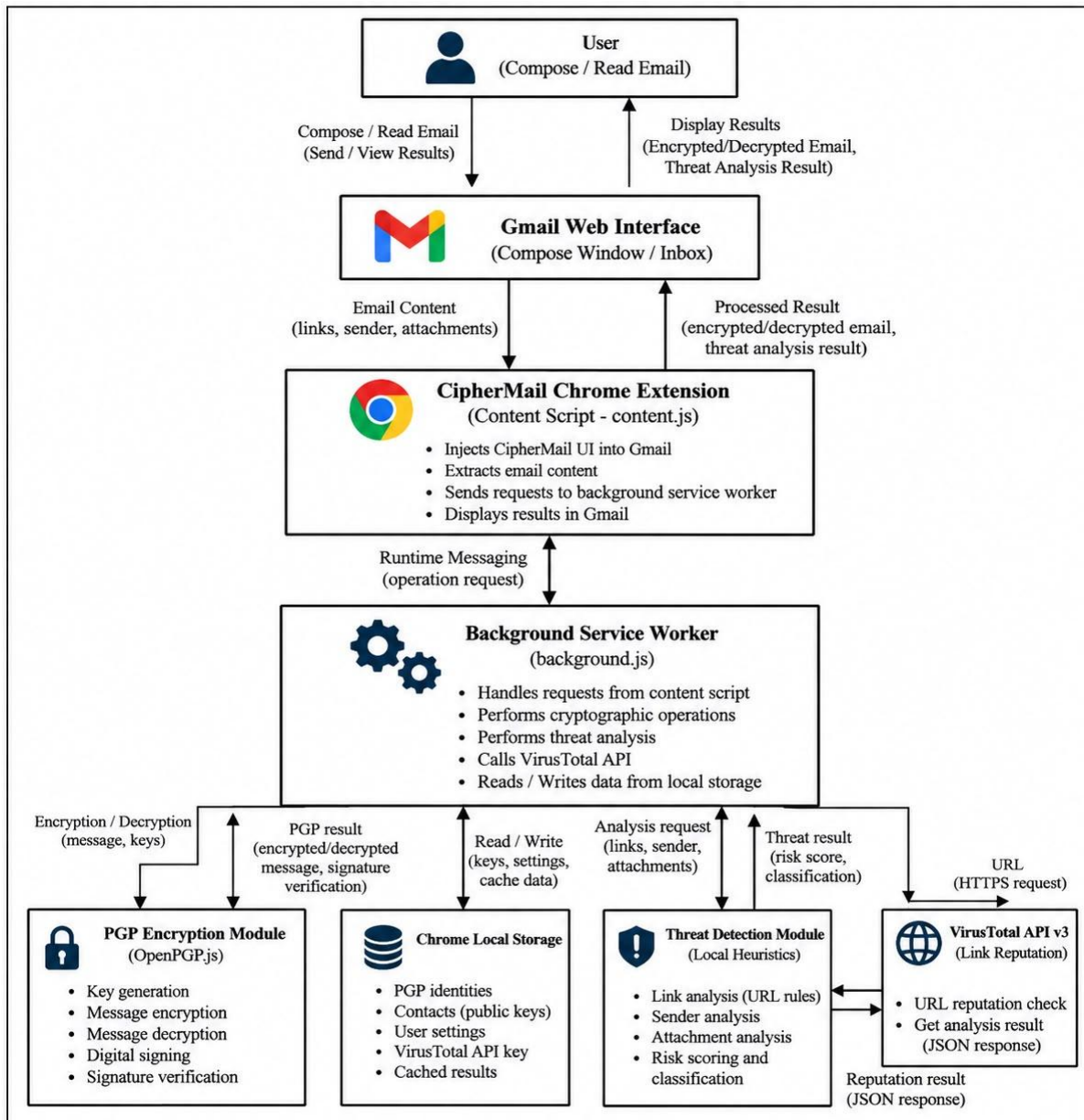


Figure 1.1 Overall proposed project architecture

CHAPTER 2

LITERATURE SURVEY

2.1 LITERATURE REVIEW

Author(s) / Year	Title of the Paper	Methodology / Approach	Inference
Poddebniak et al., 2018	Efail: Breaking S/MIME and OpenPGP Email Encryption using Exfiltration Channels	Security analysis of OpenPGP and S/MIME with email clients.	Shows that secure email requires not only strong cryptography but also secure handling of decrypted content and email rendering.
Sahoo, Liu & Hoi, 2017	Malicious URL Detection using Machine Learning: A Survey	Survey of machine-learning methods, URL features and detection techniques for malicious URLs.	Shows that URL-based detection is important for identifying phishing and malicious websites.
Kalaharsha & Mehtre, 2021	Detecting Phishing Sites – An Overview	Reviews blacklist, whitelist, content-based, URL-based, visual-similarity and machine-learning methods.	Demonstrates that phishing can be detected using multiple indicators.
Authors, 2022	Survey on Malicious URL Detection Techniques	Reviews blacklist-based, rule-based, ML and deep-learning techniques for malicious URL detection.	Identifies phishing URLs, redirects, URL shortening and social engineering as important threats.
Authors, 2022	A Comparative Analysis of Machine Learning-Based Website Phishing Detection Using URL Information	Compares Decision Tree, Random Forest, KNN, Gaussian Naive Bayes and XGBoost using URL features.	Shows that URL characteristics can distinguish legitimate and phishing websites.
Authors, 2022	Phishing Websites Detection using Machine Learning with URL Analysis	Uses URL datasets and extracts URL features for classification using multiple ML algorithms.	Demonstrates the usefulness of URL features for phishing detection and future ML enhancement.

Authors, 2022	Phishing Detection Leveraging Machine Learning and Deep Learning: A Review	Reviews ML and deep-learning models for phishing detection.	Shows that advanced learning techniques can improve automated phishing detection.
Authors, 2022	Accuracy Comparison of Different Machine Learning Models in Phishing Detection	Compares Naive Bayes, KNN, Random Forest, Decision Tree, SVM and Logistic Regression.	Demonstrates that different ML algorithms provide different detection performance.
Authors, 2023	Exploring Techniques for Web Phishing Detection: A Comprehensive Survey	Reviews phishing attacks and URL detection using ML and deep learning.	Supports detecting malicious URLs before users interact with them.
Authors, 2023	Phishing Detection System Through Hybrid Machine Learning Based on URL	Combines multiple ML algorithms with feature selection and ensemble techniques.	Shows that hybrid detection techniques can improve phishing classification.
Authors, 2023	Phishing Detection Using Machine Learning Techniques	Evaluates KNN, Decision Tree, Random Forest, Logistic Regression, CNN and RNN.	Shows that ML can automatically classify malicious and legitimate URLs.
Authors, 2023	Social Engineering Threat: Phishing Detection using Machine Learning Approach	Studies phishing as a social-engineering attack using ML-based URL detection.	Highlights the importance of detecting deceptive messages and malicious links.
Authors, 2023	Review Paper Real-Time Phishing Website With Machine Learning	Reviews real-time phishing detection using URL and website features.	Emphasizes real-time detection for protecting users before sensitive information is submitted.
Authors, 2025	Phishing Detection: A Comprehensive Survey of Methods and Trends	Reviews phishing detection across email, SMS, social media, QR codes and other channels using ML, NLP and behavioral approaches.	Identifies the need for real-time, user-centered and lightweight detection.
Chakravorty et al., 2026	Context-Aware Phishing Email Detection Using Machine Learning and NLP	Uses contextual email information, ML and NLP for phishing email detection.	Provides a future direction for incorporating email-content and contextual analysis.

Table 2.1 Comparison of Reviewed Literature

2.2 COMPARISON AND DISCUSSION

Aspect	Existing Approaches	Proposed System
Primary focus	Often focus on either email confidentiality or phishing detection.	Combines local PGP email privacy and inbox-level threat screening in one extension.
Encryption	Dedicated email clients or separate encryption tools are common.	PGP encryption and signing controls are injected into Gmail's compose workflow.
Threat detection	May use blacklists, ML, URL features or reputation services.	Uses local heuristics plus VirusTotal URL reputation.
User workflow	Security may require moving between tools.	Shield scan is available in Gmail's existing email hover toolbar.
Architecture	Some solutions use application servers or centralized services.	No CipherMail backend; browser-local processing plus direct VirusTotal requests.
Storage	May rely on remote accounts/databases.	Uses chrome.storage.local for identities, contacts, settings, API key and cached results.
Attachment handling	May support integrated encrypted attachment workflows.	Manual attachment encryption is provided; automatic Gmail attach interception is not implemented.
Limitations	Varies by tool and implementation.	Depends on Gmail DOM structure, VirusTotal quotas and browser-local security.

Table 2.2 Comparison of Existing Approaches and Proposed System

2.3 RESEARCH GAP AND CONCLUSION

The reviewed work shows strong development in both email encryption and phishing detection, but these functions are frequently treated as separate problems. PGP provides confidentiality and authenticity mechanisms, while phishing detection focuses on identifying deceptive messages or URLs. A browser extension can connect these functions at the point where users already interact with their email.

The project addresses this practical gap by combining a local PGP workflow with an inbox threat-scan workflow. The design does not attempt to replace advanced phishing-detection research. Instead, it uses transparent local rules and a reputation service to provide an additional pre-action warning mechanism.

The research also highlights limitations that are relevant to the project. Secure email implementations must consider message rendering and integration behaviour. URL-only detection cannot capture every form of phishing, while machine-learning systems introduce datasets, model maintenance and false-positive/false-negative considerations. These observations motivate future work such as content-aware analysis, stronger attachment reputation checks and more resilient Gmail integration.

CHAPTER 3

SYSTEM DESIGN

3. SYSTEM DESIGN

The system design defines how Gmail, the Chrome extension content script, the background service worker, the cryptographic layer, local storage and VirusTotal interact. The architecture is intentionally separated into modules so that Gmail DOM handling does not perform cryptographic operations directly.

3.1 SYSTEM ARCHITECTURE

The Gmail Interface Layer is the user-facing part of the system. The content script observes Gmail's dynamically generated compose windows and email rows, adds Encrypt/Sign and Decrypt controls where required, and inserts the CipherMail shield into the email hover toolbar.

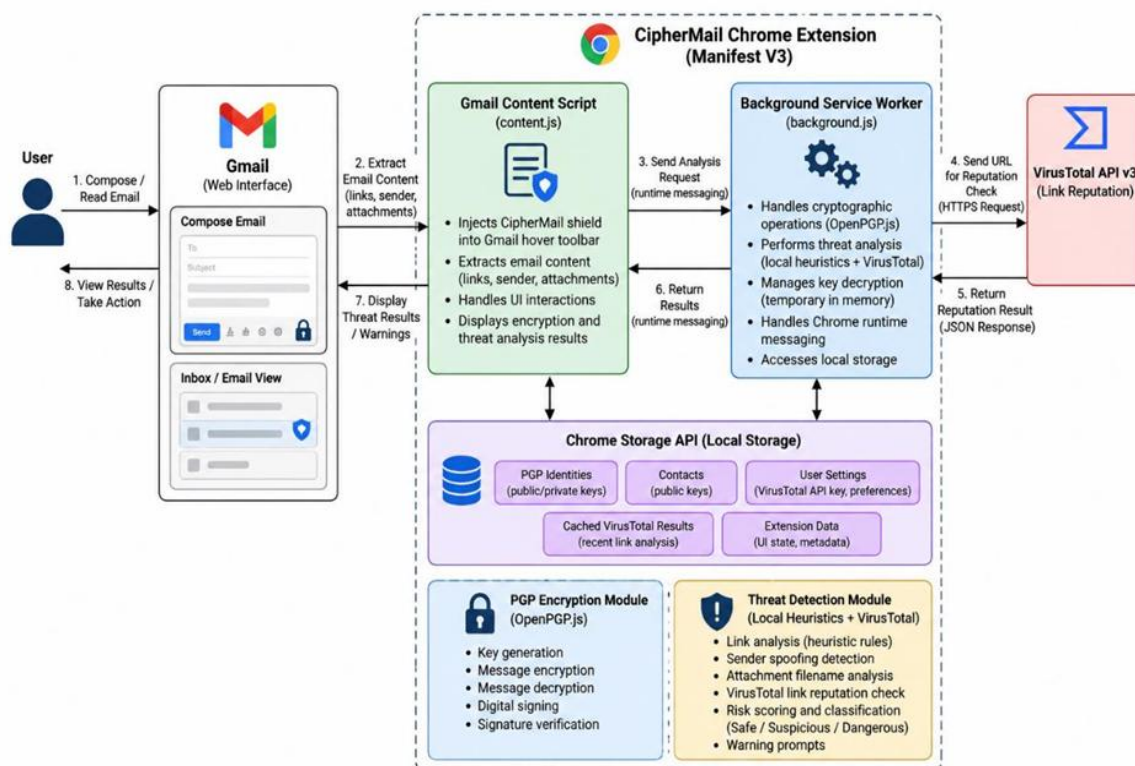
The Extension Messaging Layer uses chrome.runtime messaging to transfer requests between the Gmail content script and the background service worker. This is important because sensitive operations and external API calls are kept outside the Gmail page context.

The Cryptography Layer contains crypto/pgp.js and crypto/key-manager.js. The PGP wrapper is the only module that directly calls OpenPGP.js. Key-management functions combine cryptographic operations with browser-local storage to manage identities and contacts.

The Threat-Analysis Layer contains `security/email-analyzer.js` and `security/virustotal.js`. The analyzer applies local rules to links, sender information and attachment names. The VirusTotal client performs URL reputation lookups, caches results and handles rate-limit conditions.

The Storage Layer uses `chrome.storage.local`. The report specifies that PGP identities, contacts, settings, the user's VirusTotal API key and cached VirusTotal results are stored locally. Passphrases themselves are not stored; private keys are retained in encrypted form.

Figure 3.1 System Architecture



3.2 SYSTEM REQUIREMENTS

The project requires a desktop system capable of running Google Chrome and the extension, a Gmail account for integration testing, Node.js for the automated test suite, and network connectivity when VirusTotal reputation checks are required.

- Chrome browser with Manifest V3 extension support.
- Node.js environment supporting node:test.
- OpenPGP.js 6.3.1 dependency used by the extension.
- Gmail account for testing the content-script integration.
- Internet connection for VirusTotal API requests.
- VirusTotal API key supplied by the user for threat-reputation checks.
- Code editor and version-control environment for development.

Software / Tool	Requirement	Purpose
Google Chrome	Manifest V3 compatible browser	Hosts and executes the extension.
JavaScript	ES Modules	Implements extension logic.
OpenPGP.js 6.3.1	Installed project dependency	Key generation, encryption, decryption, signing and verification.
Node.js	node:test support	Runs automated crypto-layer tests.
Gmail	Webmail interface	Target platform for compose, decryption and threat scanning.
VirusTotal API v3	User API key	URL reputation lookup.
HTML / CSS	Browser extension pages	Popup, settings and onboarding interfaces.

Table 3.1 Software Requirements

Hardware	Recommended Requirement	Purpose
Computer / Laptop	Modern desktop/laptop	Runs Chrome, development tools and the extension.
CPU	Multi-core processor	Supports browser and development workloads.
RAM	Adequate memory for Chrome and development tools	Maintains browser, editor and test processes.
Network	Internet access when required	Supports Gmail and VirusTotal API requests.
Storage	Sufficient local storage	Stores project source, dependencies and browser extension data.

Table 3.2 Hardware Requirements

DATA ITEM	STORAGE / TRANSFER	PURPOSE
PGP identity	chrome.storage.local	Stores identity information and encrypted private-key material.
Contact public key	chrome.storage.local	Enables encryption for known recipients.
Settings	chrome.storage.local	Theme, timeout and user configuration.
VirusTotal API key	chrome.storage.local	Authenticates the user's reputation requests.
Cached VirusTotal results	chrome.storage.local	Reduces repeated API calls.
Email body	Browser processing	Used for encryption/decryption; not sent to CipherMail backend.
Link URL	Sent to VirusTotal when reputation lookup is requested	External reputation analysis.

Table 3.3 Vehicle-State Parameters

3.3. DATA FLOW AND ACTIVITY DESIGN

The flow starts with Gmail. For encryption, the content script reads the message, gets the recipient's public key, and sends it to the background worker for PGP encryption/signing. The encrypted message is returned to Gmail for sending.

For threat detection, the content script extracts links, sender details, and attachments. Local rules and VirusTotal analyze the data, and the combined threat score is shown in Gmail.

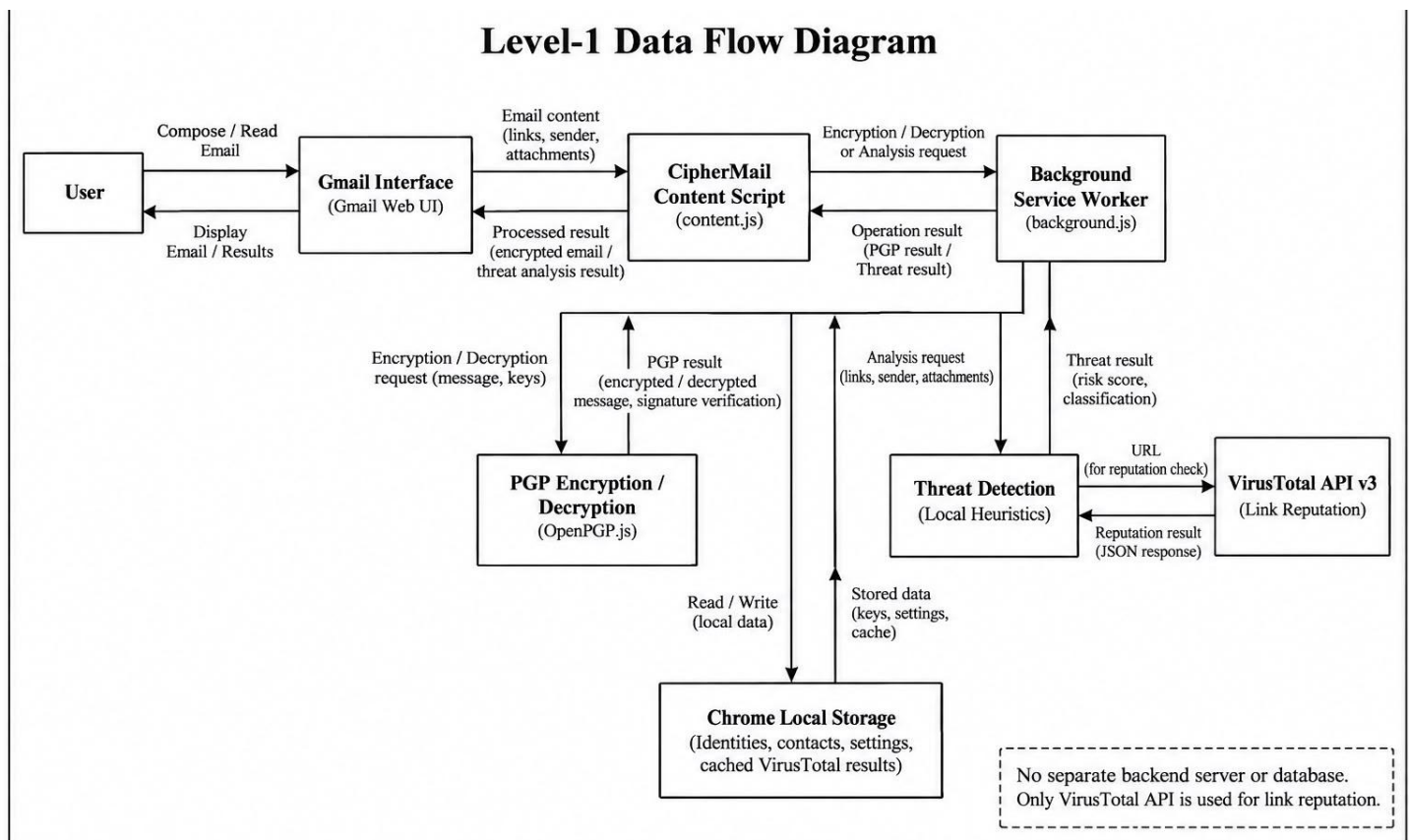


Figure 3.2 Level-1 Data Flow Diagram

The activity sequence begins when Gmail loads and the extension observes the page. The extension identifies relevant compose windows and email rows, adds the required controls, waits for user interaction and then executes the selected workflow. After the result is displayed, the system returns to the waiting state.

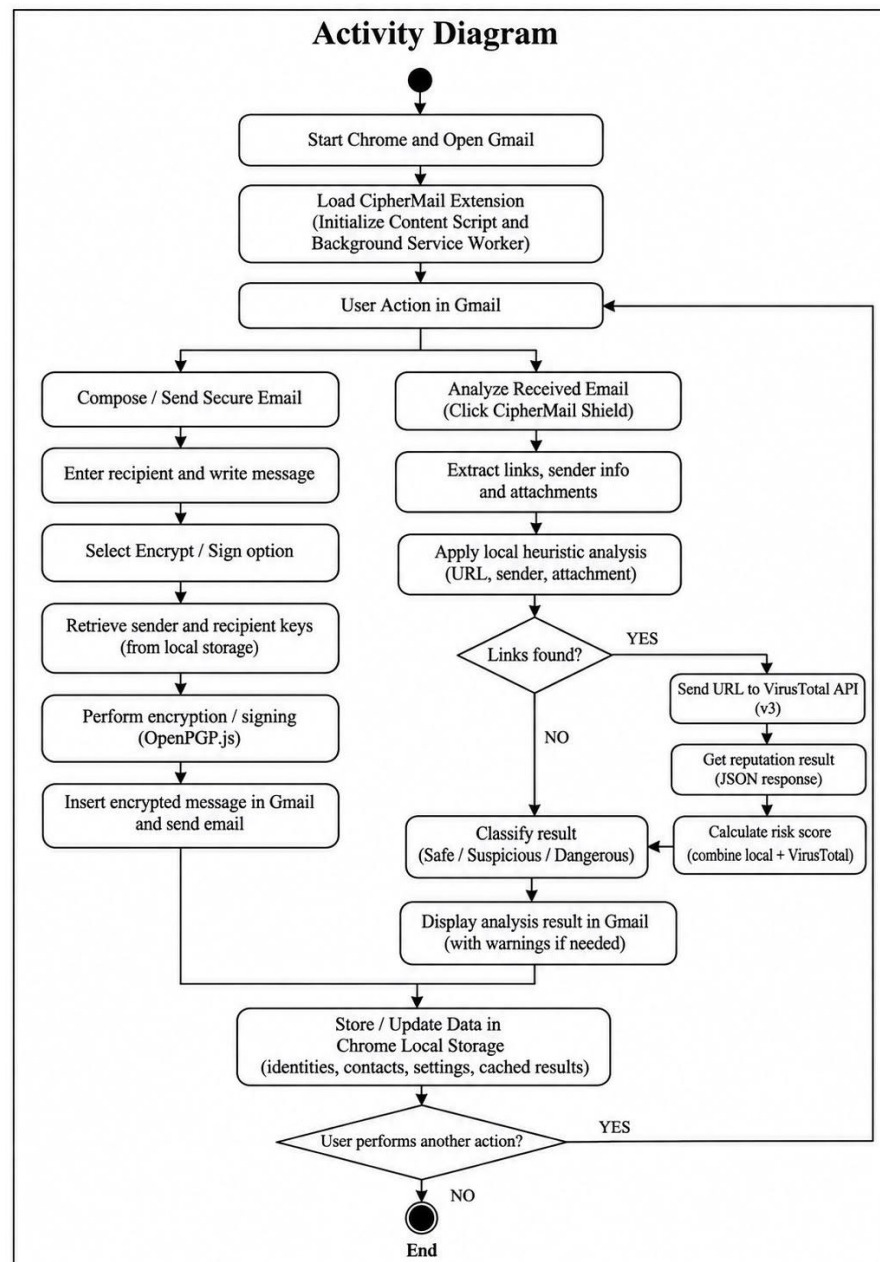


Figure 3.3 Activity Diagram

3.4 DEPLOYMENT AND SCALING

The extension is deployed locally as a Chrome Manifest V3 extension. There is no CipherMail application server. Cryptographic operations occur in the browser through OpenPGP.js, while VirusTotal requests are made directly from the extension's background service worker.

Security design includes separating DOM interaction from cryptographic logic, keeping private keys encrypted at rest, decrypting private keys only briefly in memory during an operation, inserting decrypted message content using `textContent` rather than `innerHTML`, avoiding `eval()`, and using a restrictive Content-Security-Policy.

The architecture has practical deployment limitations. Gmail's HTML structure is unofficial and can change, so selectors may require maintenance. VirusTotal usage depends on internet connectivity and API quotas. The threat detector adds a warning/confirmation step but cannot override Gmail's own native click or attachment handling.

CHAPTER 4

PROJECT DESCRIPTION

The CipherMail is a browser-based security extension that integrates privacy protection and threat detection into Gmail. The methodology begins with local key management, continues through PGP encryption and decryption, and adds a second workflow for inspecting potentially dangerous email content.

4.1 METHODOLOGIES

- **Browser Extension Methodology:** Implement the project as a Chrome Manifest V3 extension using content scripts, a background service worker and extension pages.
- **PGP Key Management:** Generate, import, export and locally store public/private key material using OpenPGP.js and chrome.storage.local.
- **Hybrid Encryption:** Encrypt message content with a randomly generated session key and protect the session key with the recipient's public key.
- **Digital Signing:** Optionally sign messages with the sender's private key so the recipient can verify the signature.
- **Gmail Integration:** Detect compose windows and opened emails and inject project controls into Gmail's UI.
- **Local Threat Heuristics:** Inspect link structure, sender/domain relationship and attachment file names for known warning indicators.

- Risk Classification: Map combined evidence to Safe, Suspicious or Dangerous.
- Warning-Oriented Prevention: Display a confirmation warning before opening flagged links or downloading flagged attachments.
- Automated Testing: Use Node.js node:test for automated tests covering the cryptographic layer.

4.2 MODULES

The prototype system relies on a modular architecture to handle secure email communication, threat detection, and Gmail integration:

- PGP & Key Management Module: Generates and manages PGP keys and handles message encryption, decryption, and digital signing.
- Gmail Integration Module: Provides the interface for reading, encrypting, decrypting and sending emails directly within Gmail.
- Security & Threat Detection Module: Analyzes links, sender information, and attachments using local rules and VirusTotal to identify potential threats.
- Background & Storage Module: Manages message routing, key storage, and communication between the extension components.
- Attachment & Testing Module: Handles secure attachment encryption/decryption and performs automated tests to verify system functionality.

4.3 MODULE INTERACTION

The modules operate as a controlled pipeline. Gmail is accessed only through the content script. The content script communicates with the background service worker using Chrome runtime messaging. The background worker invokes cryptographic or security modules as required.

For encryption, the content script supplies message content and recipient information. The key manager identifies the recipient public key and the PGP wrapper performs encryption and optional signing. The result is returned to Gmail.

For threat scanning, the content script supplies extracted links, sender details and attachment names. The analyzer calculates local risk indicators. The VirusTotal module checks link reputation and returns the available reputation data. The analyzer combines the evidence and sends the final classification to the Gmail results panel.

CHAPTER 5

IMPLEMENTATION, RESULTS AND DISCUSSION

The implementation phase converts the proposed architecture into a working Chrome extension integrated with Gmail. The implemented system covers the privacy workflow, threat-detection workflow, local storage and automated testing. The project documentation states that both major parts are implemented.

5.1 IMPLEMENTATION RESULTS

5.1.1 PGP KEY MANAGEMENT AND ENCRYPTION

The key-management workflow creates a PGP identity containing a name, email address and passphrase-protected private key. The extension uses ECC/Curve25519 key material as documented in the project. Public keys can be shared with contacts, while private-key material remains local and encrypted at rest.

OpenPGP.js performs the cryptographic operations. The implementation uses a hybrid encryption process: a random session key is used for efficient message encryption, and the session key is protected using the recipient's public key. This allows large message content to be processed efficiently while retaining public-key confidentiality.

- Generate PGP key pair.
- Store protected private-key material locally.
- Import existing keys.
- Export or back up keys as .asc data.

- Store recipient public keys as contacts.
- Encrypt messages for a selected recipient.
- Decrypt received PGP messages.

5.1.2 GMAIL COMPOSE ENCRYPTION

The Gmail content script observes compose windows and provides Encrypt and Sign controls. When the user activates encryption, the extension reads the message content and identifies the recipient. The background service worker performs the requested cryptographic operation and returns the PGP result.

The project documentation describes this as an integrated Gmail workflow rather than a separate application. The encrypted PGP block is placed into the message body so Gmail can transmit it using its normal email delivery process.

5.1.3 MESSAGE DECRYPTION AND SIGNATURE VERIFICATION

When a user opens an email containing a PGP block, the extension can detect the encrypted content and provide a decryption action. The private key is obtained through the key-management process and is decrypted briefly in the background service worker's memory during the operation.

After successful decryption, the plaintext is inserted into the page using `textContent`. The project documentation explicitly identifies this as a protection against treating message content as HTML. Signature verification status is also displayed so that the user can distinguish confidentiality from message authenticity.

5.1.4 THREAT DETECTION AND RISK SCORING

The threat-detection feature is activated through the CipherMail shield placed beside Gmail's Archive, Mark as read and Delete actions. When clicked, the content script extracts links, sender display name and actual domain information, and attachment file names.

CHECK	IMPLEMENTATION
Visible text vs. actual href	Compares displayed link text with its actual destination.
IP-address URLs	Flags direct IP-address destinations as a risk indicator.
Look-alike domains	Identifies domain patterns that resemble another domain.
URL shorteners	Flags shortened links because the final destination is hidden.
Non-HTTPS links	Adds risk for links that do not use HTTPS.
Sender spoofing	Compares display name with the actual sender domain.
Dangerous extensions	Flags risky extensions such as .exe, .scr and .js.
Double extensions	Flags patterns such as .pdf.exe.
VirusTotal reputation	Adds reputation-based evidence for each checked URL.
Final risk score	Maps combined evidence to Safe, Suspicious or Dangerous.

5.2 DISCUSSION

The prototype demonstrates the integration of two security workflows in the same Gmail environment. The privacy component focuses on message confidentiality and optional signing, while the threat component focuses on pre-action awareness for links, senders and attachments.

The project's no-backend design reduces the need to operate a separate CipherMail server. Local storage is sufficient for identities, contacts, settings and cached results. The main external dependency is VirusTotal for URL reputation. This architecture also makes the privacy boundary easier to describe: the project itself does not receive email content through a custom server.

The threat detector is intentionally rule-based rather than AI-based. This makes the current implementation transparent and lightweight, but it also limits its ability to identify sophisticated attacks that do not exhibit the selected indicators. Advanced ML/NLP-based phishing detection can therefore be considered future work.

Figure 5.1 CipherMail Setup



Figure 5.2 CipherMail Interface

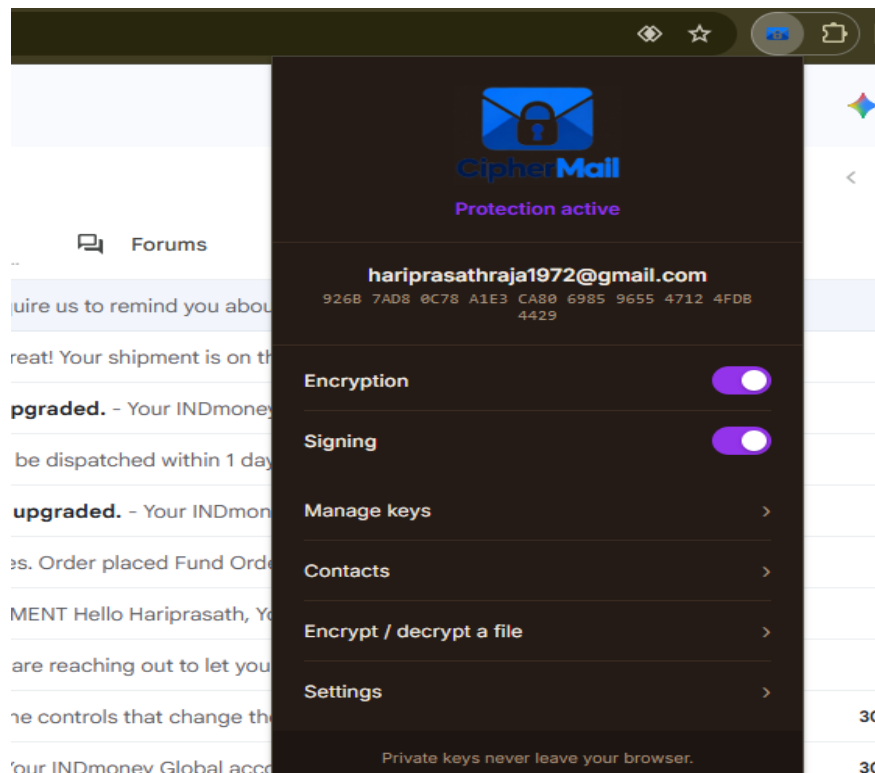


Figure 5.3 Encrypted Email Composition Interface

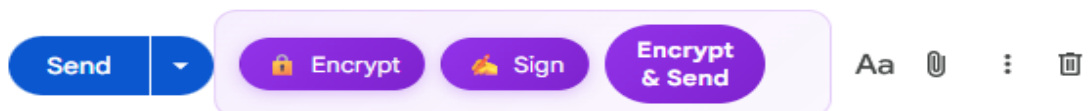


Figure 5.4 PGP Key Unlock Interface

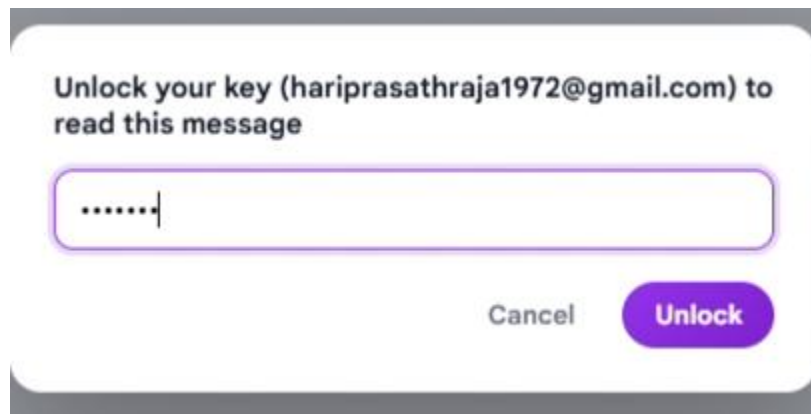


Figure 5.5 PGP Message Decryption Interface

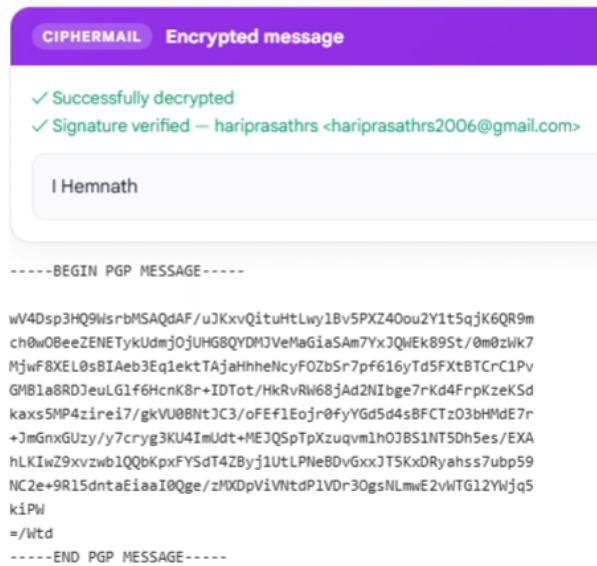


Figure 5.6 Email Threat Analysis Results

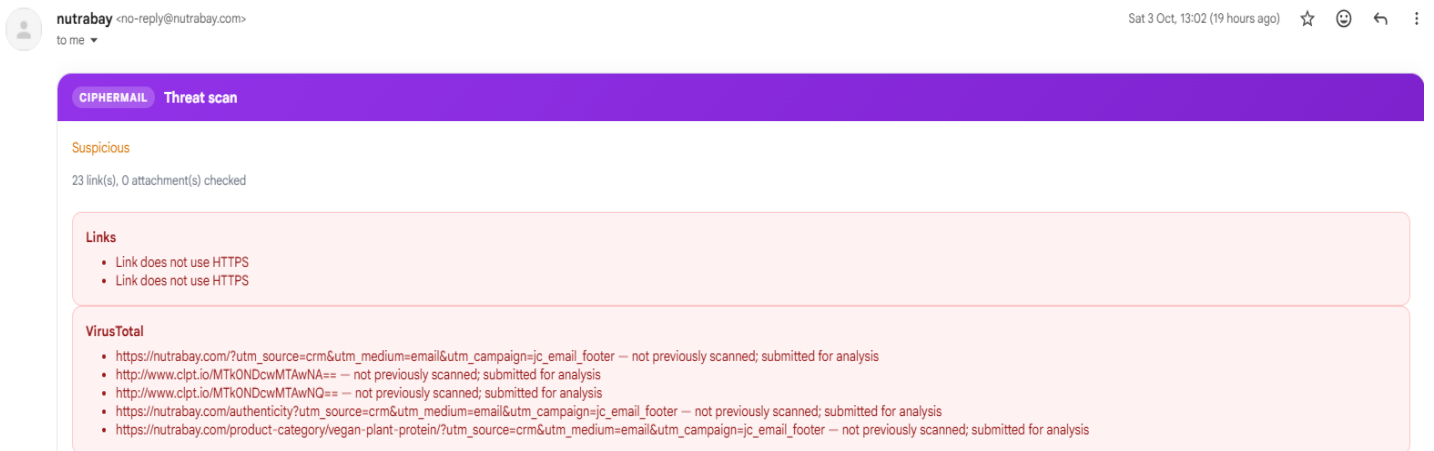
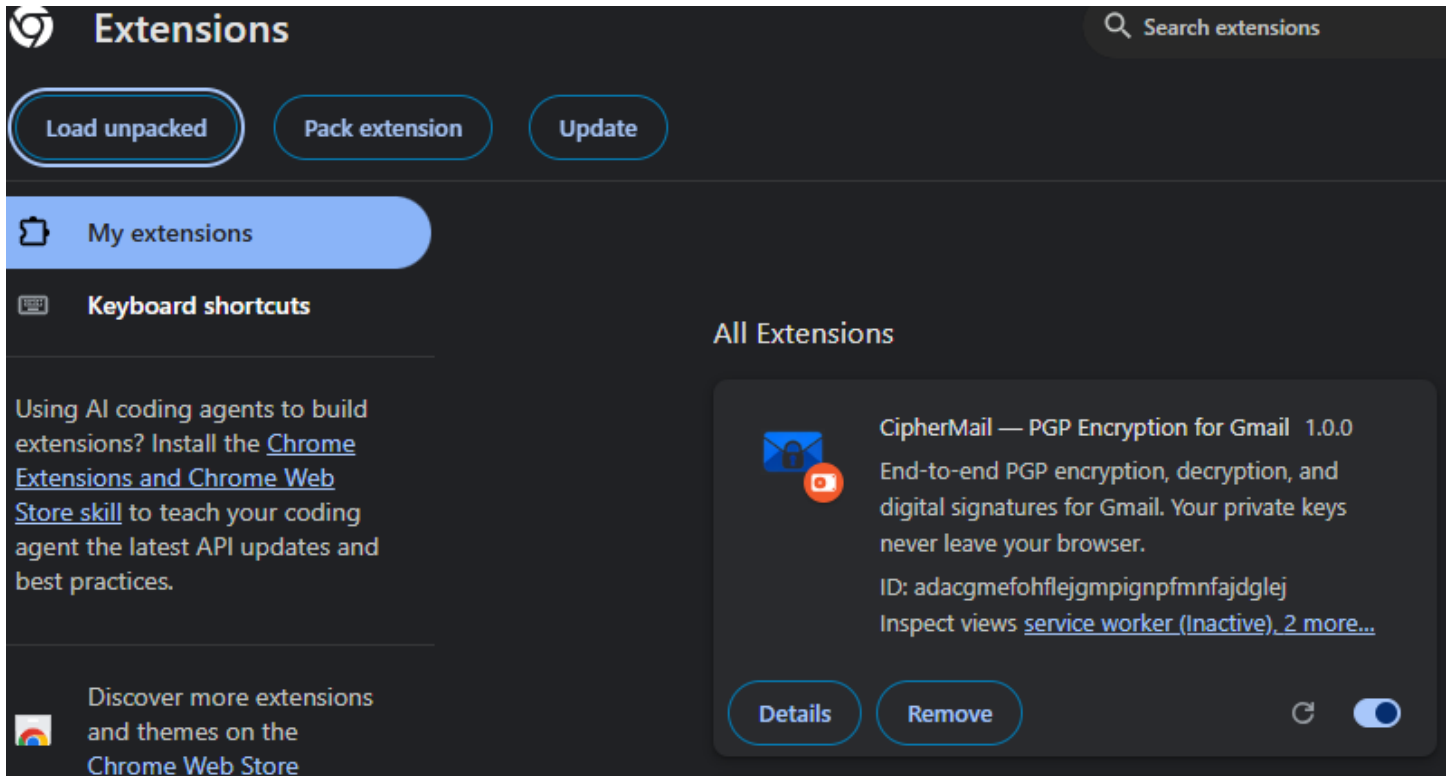


Figure 5.7 CipherMail Chrome Extension Configuration



5.3 TEST SCENARIOS AND PROTOTYPE STATUS

Testing is organized around the major implemented workflows. The project documentation states that 50 automated tests cover the crypto layer. Functional testing can additionally be organized into key management, message encryption/decryption, signature verification, Gmail integration and threat-analysis scenarios.

Component	Current Status	Implementation / Scope
Key generation	Implemented	PGP identity generation with name, email and passphrase.
Key import/export	Implemented	Existing keys can be imported and backed up/exported.
Contact management	Implemented	Recipient public keys stored in a searchable contact list.
Compose encryption	Implemented	Encrypt/Sign controls integrated into Gmail compose.
Message decryption	Implemented	PGP blocks detected in opened messages and decrypted locally.
Attachment encryption	Implemented	Separate manual drag-and-drop encryption/decryption tool.
Threat shield	Implemented	Shield added to email hover toolbar.
Phishing/suspicious URL checks	Implemented	Local rules plus VirusTotal URL reputation.
Sender spoofing checks	Implemented	Display-name and domain comparison.
Dangerous attachment checks	Implemented	Extension and double-extension checks are local.
Risk classification	Implemented	Safe / Suspicious / Dangerous results.

Table 5.1 Prototype Implementation Status

Test Scenario	Condition / Input	Expected Behaviour	Current Status
Key generation	Create a new identity	PGP key pair is generated and stored in protected form..	Demonstrated
Message encryption	Compose Gmail message and select Encrypt	Plaintext is replaced with a PGP block.	Demonstrated
Message decryption	Open a received PGP message	Extension detects the block and provides decrypted content.	Prototype / mocked
Suspicious URL	Email contains a risky URL pattern	Local rules add risk evidence.	Demonstrated
VirusTotal lookup	Email contains a URL and API quota is available	URL reputation is retrieved and combined with local evidence.	Demonstrated
Dangerous attachment	Attachment name uses risky/double extension	Warning is displayed before download.	Demonstrated

Table 5.2 Test Scenarios and Expected Behaviour

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

CipherMail demonstrates a combined privacy and threat-detection approach for Gmail using a Chrome Manifest V3 extension. The project implements PGP-based key management, encryption, decryption, signing and verification, together with a Gmail-integrated threat-scan action.

The privacy workflow is implemented locally using OpenPGP.js. The project does not implement cryptographic mathematics itself; instead, it uses a dedicated cryptographic library wrapper. Private keys remain encrypted at rest and are decrypted only briefly in memory when an operation requires them.

The threat-detection workflow provides local heuristic analysis of URLs, sender information and attachment names, supplemented by VirusTotal reputation checks for links. Results are combined into Safe, Suspicious or Dangerous classifications, and the user receives a warning before flagged actions.

The no-backend architecture is a significant design characteristic of the project. There is no CipherMail server or project-specific database. Browser-local storage is used for identities, contacts, settings, API keys and cached reputation results. The main external data transfer is the link URL sent to VirusTotal for reputation analysis.

The project also establishes clear boundaries. It does not claim protection against a compromised device, automatic interception of Gmail's native attachment upload flow, perfect phishing detection or a true hard block of Gmail's native actions. These limitations are explicitly considered part of the prototype's security and deployment model.

6.2 FUTURE WORK

- Develop a more resilient Gmail integration layer that can tolerate changes to Gmail's DOM structure.
- Support automatic attachment encryption within the Gmail attachment workflow if a reliable integration mechanism becomes available.
- Support true PGP/MIME packaging so encrypted attachments can be integrated into standards-based email structures.
- Add keyserver or other controlled public-key discovery mechanisms to reduce manual key exchange.
- Extend threat detection with content-aware phishing analysis and machine-learning/NLP models.
- Add richer domain and URL intelligence, including redirects and contextual analysis.
- Support VirusTotal file-hash or other safe attachment reputation workflows where appropriate.
- Improve attachment processing for large files using streaming rather than loading the complete file into memory.
- Strengthen protection of sensitive extension settings and API credentials.
- Evaluate false positives and false negatives using a controlled test corpus.
- Add performance measurements for encryption time, decryption time, URL analysis latency and cache effectiveness.

REFERENCES

- [1] D. Poddebniak et al., “Efail: Breaking S/MIME and OpenPGP Email Encryption using Exfiltration Channels,” USENIX Security Symposium, 2018.
- [2] D. Sahoo, C. Liu, and S. C. H. Hoi, “Malicious URL Detection using Machine Learning: A Survey,” arXiv:1701.07179, 2017.
- [3] P. Kalaharsha and B. M. Mehtre, “Detecting Phishing Sites – An Overview,” arXiv:2103.12739, 2021.
- [4] “Survey on Malicious URL Detection Techniques,” 2022 6th International Conference on Trends in Electronics and Informatics (ICOEI), IEEE, 2022.
- [5] “A Comparative Analysis of Machine Learning-Based Website Phishing Detection Using URL Information,” 2022 5th International Conference on Pattern Recognition and Artificial Intelligence (PRAI), IEEE, 2022.
- [6] “Exploring Techniques for Web Phishing Detection: A Comprehensive Survey,” 2023 7th International Conference on Intelligent Computing and Control Systems (ICICCS), IEEE, 2023.
- [7] “Across the Spectrum: In-Depth Review AI-Based Models for Phishing Detection,” IEEE Open Journal of the Communications Society, vol. 6, pp. 2065–2089, 2024.
- [8] “Study and Analysis of Machine Learning Models for Detection of Phishing URLs,” Wiley-IEEE Press, 2024, pp. 85–95.
- [9] “Phishing URL Detection Using Machine Learning: A Comparative Study,” 2025 6th International Conference on Mobile Computing and Sustainable Informatics (ICMCSI), IEEE, 2025.
- [10] “Email Phishing Detection Using Machine Learning,” 2025 IEEE 3rd International Symposium on Sustainable Energy, Signal Processing and Cybersecurity, IEEE.

- [11] A. Chakravorty, M. Price, N. Elsayed, and Z. ElSayed, “Context-Aware Phishing Email Detection Using Machine Learning and NLP,” arXiv:2603.27326, 2026.
- [12] OpenPGP.js Documentation, OpenPGP.js v6.3.1, project documentation.
- [13] VirusTotal Documentation, “Browser Extensions” and VirusTotal API documentation.
- [14] Chrome Developers, Chrome Extensions documentation, Manifest V3, content scripts, service workers and storage APIs.

APPENDICES

APPENDIX A — SOFTWARE AND HARDWARE REQUIREMENTS

Category	Item	Requirement / Specification	Purpose
Software	Google Chrome	Manifest V3 compatible	Runs the extension.
Software	JavaScript	ES Modules	Extension logic.
Software	OpenPGP.js	v6.3.1	Cryptographic operations.
Software	Node.js	node:test	Automated crypto tests.
Software	Gmail	Web interface	Target email environment.
Software	VirusTotal API	API v3 + user key	URL reputation.
Hardware	Computer/Laptop	Modern system	Development and execution.
Hardware	Network	Internet when required	Gmail/VirusTotal connectivity

APPENDIX B — KEY SOURCE FILES AND FUNCTIONS

FILE	PURPOSE	KEY FUNCTIONS / RESPONSIBILITY
crypto/pgp.js	PGP wrapper	generateKeyPair, encryptMessage, decryptMessage and related operations.
manifest.json	Extension configuration	Permissions, entry points and security configuration.
crypto/key-manager.js	Identity/contact layer	createIdentity, importContactKey and key-management operations.
background/service-worker.js	Background message router	handleMessage, encryptForGmail and VirusTotal requests.
content/gmail.js	Gmail integration	handleComposeWindow, handleDecryptClick and shield injection.
storage/storage.js	Local persistence	saveIdentity, saveContact and settings/cache storage.
email/attachment-handler.js	Attachment operations	encryptAttachment and decryptAttachment.
security/email-analyzer.js	Threat rules	Link, sender and attachment checks plus risk score.

security/virustotal.js	VirusTotal client	URL lookup, result cache and quota handling.
tests/*.test.js	Automated tests	Crypto-layer test suite.

APPENDIX C – SYSTEM MODULES AND FUNCTIONS

Module	Main Function	Current Phase I Status	Future Extension
Identity / Key Module	Creates and manages PGP identities	Implemented	Improved key discovery and recovery workflows.
Compose Module	Encrypts/signs Gmail messages	Implemented	More resilient Gmail integration.
Decrypt Module	Detects and decrypts PGP blocks	Implemented	Improved rendering and compatibility.
Threat Analyzer	Local link/sender/attachment checks	Implemented	Context-aware ML/NLP detection.
VirusTotal Module	URL reputation lookup	Implemented	Broader reputation and safe attachment intelligence.
Warning Module	Pre-action warning	Implemented	Stronger browser-level intervention where technically supported.
Attachment Module	Manual file encryption/decryption	Implemented	PGP/MIME and integrated attachment workflow.

